Managing releases and in-app updates

This guide is for **maintainers** who ship new versions of the Code Companion **Electron** desktop app and need **Software Updates** (electron-updater + GitHub Releases) to keep working as the product evolves.

For **how to build** installers locally, see **BUILD.md**.

Goals

- **Predictable versions** — one source of truth (`package.json`) aligned with git tags.
- **Complete GitHub Releases** — every published version includes updater metadata and binaries so users are not stuck on 404s for `latest-*.yml`.
- **Repeatable process** — prefer CI on tag push; use local publish only when needed.

Concepts

Piece	Role
`package.json` → `version`	Semver string shipped inside the app; must match the release tag (without leading `v`).
Git tag `v*`	Triggers CI and names the GitHub Release (e.g. `v1.6.0` → version `1.6.0`).
`electron-builder.config.js` → `publish`	Points electron-updater at **GitHub** `owner` / `repo` (currently `th3rdai` / `CodeCompanion`).
`release/` outputs	Per-platform artifacts plus **updater YAML** (e.g. macOS `latest-mac.yml`). These must be **attached to the same GitHub Release** the app resolves.
`electron/updater.js`	Configures `autoUpdater` (including **`allowPrerelease`**). See Prereleases.

Before you cut a release

1. **Land changes on the main branch** (or the branch you release from) and run checks you care about (e.g. `npm run test:unit` , targeted Playwright, manual smoke of the Electron build).
2. **Bump `version`** in `package.json` using semver conventions you adopt for this project (e.g. patch for fixes, minor for features, major for breaking changes).
3. **Commit** the version bump (and changelog notes if you keep them in-repo).

4. **Tag** using the `v` **+ version** pattern so CI can validate:
   `v1.6.0` ↔ `"version": "1.6.0"`.

The GitHub Actions workflow **fails** if the tag suffix does not match `package.json` — that is intentional.

Recommended path: CI (tag push)

Workflow: `.github/workflows/build.yml`

1. Ensure `package.json` has the new version committed.
2. Create and push an annotated or lightweight tag:
   `git tag v1.6.0`
   `git push origin v1.6.0`
3. CI builds **macOS**, **Windows**, and **Linux**, then creates a **single GitHub Release** and uploads **all** `release/` outputs from each job (including `latest-mac.yml` and blockmaps where produced).

Notes

- **Manual "Run workflow"** from the Actions UI only runs builds and stores **workflow artifacts**. It does **not** create a GitHub Release — users relying on **Software Updates** still need a **tag push** (or a manual publish; see below).
- Tags whose names include `-beta`, `-alpha`, or `-rc` are marked **prerelease** on GitHub.
- Artifacts are downloaded into **separate folders** before upload so platform-specific files are not overwritten.

Alternative: publish from one machine

Use when CI is unavailable or you need to ship a hotfix build quickly.

Requirements

- `GH_TOKEN` (or `GITHUB_TOKEN`) with permission to create releases and upload assets to the target repo.
- `npm run build` already run by the publish scripts below.

Commands (from repo root):

```
export GH_TOKEN=ghp_...   # or: export GITHUB_TOKEN=...

npm run electron:publish:mac     # macOS only
```

```
npm run electron:publish:win     # Windows only
npm run electron:publish:linux   # Linux only
```

Each script runs `electron-builder … --publish always`, which creates or updates the GitHub Release and uploads artifacts for that platform.

macOS in-app updates require **`latest-mac.yml`**, DMG/ZIP, and **blockmaps** on the **same** release your users resolve as "latest." If you only publish Windows, mac users will not see a complete update feed.

What must exist on GitHub for updates to work

Symptom when something is missing: users see errors about **`latest-mac.yml`** (or similar) **404**, or **Upgrade** does nothing useful.

Rule: For each **released version**, the GitHub Release for that tag must include **all** files electron-builder emitted for that platform build — at minimum for **macOS**:

- `latest-mac.yml`
- Matching **`.dmg`** and **`*-mac.zip`** (names depend on `productName`, version, and arch)
- **`.blockmap`** files next to those binaries when builder produced them

Windows and Linux have their own `latest*.yml` naming; the same idea applies.

Quick check after publishing

- Open: `https://github.com/th3rdai/CodeCompanion/releases/latest/download/latest-mac.yml` (GitHub redirects to the latest **non-prerelease** stable; for prerelease-only scenarios, confirm the feed URL your updater uses.)
- Confirm the YAML **`version`** and **`url`** entries match assets on **that** release page.

Prereleases and `allowPrerelease`

`electron/updater.js` sets **`autoUpdater.allowPrerelease = true`** so repositories that **only** publish prereleases can still resolve an update (GitHub's **`/releases/latest`** API omits prereleases).

Tradeoffs

- If the **newest** prerelease has **incomplete assets**, users may hit broken update feeds until a **newer** release with full assets exists.
- Once you routinely ship **stable** releases with complete assets, you may set **`allowPrerelease`** **to** **`false`** so stable users are not pulled toward random betas — **test** Software Updates after

changing it.

Versioning discipline as the app evolves

- **Patch** (`1.5.1` → `1.5.2`): bugfixes, small safe changes.
- **Minor** (`1.5.x` → `1.6.0`): new features, backward-compatible behavior.
- **Major** (`1.x` → `2.0.0`): breaking UX, config, or platform requirements — call out in release notes.

Keep **Electron**, **Node** (for the embedded server), and **security-related** dependency bumps in mind when deciding semver; document anything users must do manually (e.g. new macOS permission strings).

Operational checklist (copy/paste)

```
[ ] package.json version bumped and committed
[ ] Tests / smoke pass as appropriate
[ ] Tag vX.Y.Z pushed (matches version)
[ ] CI green; GitHub Release contains platform artifacts + latest-*.yml
[ ] Spot-check latest-mac.yml (or platform feed) URL
[ ] Optional: announce or changelog entry for users
```

Related files

File	Purpose
`package.json`	App `version` ; npm scripts including `electron:publish:*`
`electron-builder.config.js`	`publish` target, artifact layout
`electron/updater.js`	Auto-updater behavior and `allowPrerelease`
`BUILD.md`	Local builds, artifact names, signing notes
`.github/workflows/build.yml`	Tag-triggered multi-platform build + release upload

User-facing workaround

If a release is broken or incomplete, users can always **download the DMG/ZIP/AppImage** from GitHub Releases and install manually until the feed is fixed.